LAPORAN IMPLEMENTASI SISTEM DETEKSI BUAH REAL-TIME BERBASIS DEEP LEARNING

PERBANDINGAN PERFORMA YOLOV8S DAN YOLO26S SERTA BROWSER DEPLOYMENT DENGAN ONNX RUNTIME WEB

COMPUTER VISION (W152500029)
Kamis, 13:15 - 15:45 | Ruang: D-206

UNIVERSITAS
MERCU BUANA

Dosen Pengampu:
Muhaimin Hasanudin, S.T, M.Kom

Disusun Oleh Kelompok 5:
Tegar Wijaya Kusuma (41523010217)
Muhammad Ihsar Fatiha (41523010202)
Ahmad Khadavi (41523010215)

UNIVERSITAS MERCU BUANA
FAKULTAS ILMU KOMPUTER
TEKNIK INFORMATIKA
2026

DAFTAR ISI

ABSTRAKSI.....3
BAB I: PENDAHULUAN.....5
a. Latar Belakang.....5
b. Rumusan Masalah.....7
c. Tujuan.....7
d. Manfaat.....8
BAB II: TINJAUAN PUSTAKA.....9
a. Penelitian Terdahulu (State of The Art).....9
b. Teori Pendukung.....11
BAB III: METODE PENELITIAN.....12
a. Dataset dan Sumber Data.....12
b. Data Preparation.....12
c. Data Cleansing.....13
d. Exploratory Data Analysis (EDA).....13
e. Feature Engineering.....13
f. Modeling.....14
g. Evaluasi Hasil.....14
h. Development (Web & Desktop Deployment).....15
Bab IV: Hasil dan Pembahasan.....15
a. Analisa Hasil Pelatihan Model dan Manajemen Suhu GPU.....15
b. Analisis Komparatif Performa Akurasi (mAP50) Per Kelas.....17
c. Analisis Kecepatan Inferensi dan Latensi (FPS).....19
d. Perancangan Sistem dan Analisis Deployment.....20
e. Visualisasi Hasil Deteksi Objek Real-Time.....22
f. Analisis Confusion Matrix.....23
g. Analisis Kurva Presisi-Recall (Precision-Recall Curve).....25
BAB IV: PENUTUP.....27
a. Kesimpulan.....27
DAFTAR PUSTAKA.....30

ABSTRAKSI

Deteksi objek buah secara real-time menjadi komponen penting dalam otomatisasi pertanian dan perdagangan pangan, namun proses pelatihan model deep learning pada perangkat dengan GPU berspesifikasi rendah sering terkendala fragmentasi memori CUDA dan panas berlebih. Penelitian ini bertujuan membandingkan kinerja model YOLOv8s dan YOLO26s serta menerapkan skema incremental training loop untuk mendukung proses pelatihan yang lebih stabil pada perangkat dengan keterbatasan sumber daya. Metode yang digunakan meliputi penyusunan dataset deteksi buah enam kelas (Apple, Banana, Grape, Orange, Pineapple, dan Watermelon), penerapan balanced stratified sampling menjadi 2.000 citra latih, pelatihan selama 50 epoch, serta evaluasi menggunakan mAP50 dan kecepatan inferensi. Model hasil pelatihan kemudian diimplementasikan pada peramban menggunakan ONNX Runtime Web dengan dynamic canvas rendering. Hasil penelitian menunjukkan bahwa YOLOv8s memberikan akurasi dan kecepatan inferensi yang lebih tinggi, sedangkan YOLO26s menawarkan ukuran model yang lebih ringkas sehingga lebih sesuai untuk deployment berbasis web. Pendekatan yang diusulkan menunjukkan bahwa pelatihan bertahap dapat menjadi solusi praktis untuk pengembangan model deteksi objek pada perangkat dengan sumber daya terbatas.

Kata Kunci: Deteksi Buah, YOLOv8s, YOLO26s, Incremental Training, ONNX Runtime Web.

ABSTRACT

Real-time fruit object detection is a crucial component in agricultural automation and food trade, but training deep learning models on devices with low-spec GPUs is often hampered by CUDA memory fragmentation and overheating. This study aims to compare the performance of the YOLOv8s and YOLO26s models and implement an incremental training loop scheme to support a more stable training process on devices with limited resources. The method used includes compiling a six-class fruit detection dataset (Apple, Banana, Grape, Orange, Pineapple, and Watermelon), applying balanced stratified sampling to 2,000 training images, training for 50 epochs, and evaluating using mAP50 and inference speed. The trained model is then implemented in a browser using ONNX Runtime Web with dynamic canvas rendering. The results show that YOLOv8s provides higher accuracy and inference speed, while YOLO26s offers a more compact model size, making it more suitable for web-based deployment. The proposed approach demonstrates that incremental training can be a practical solution for object detection model development on devices with limited resources.

Keywords: Fruit Detection, YOLOv8s, YOLO26s, Incremental Training, ONNX Runtime Web.

BAB I: PENDAHULUAN

a. Latar Belakang

Perkembangan teknologi computer vision telah mendorong otomatisasi pada berbagai sektor industri, termasuk pertanian dan perdagangan pangan. Salah satu penerapan yang berkembang pesat adalah deteksi objek buah secara real-time untuk membantu proses penyortiran, pengemasan, hingga pengawasan kualitas hasil panen. Dibandingkan metode konvensional yang masih mengandalkan pengamatan manusia, sistem berbasis deep learning mampu memberikan hasil yang lebih cepat, konsisten, dan objektif sehingga berpotensi meningkatkan efisiensi operasional.

Salah satu keluarga algoritma yang banyak digunakan dalam deteksi objek adalah YOLO (You Only Look Once). Model YOLO dirancang sebagai single-stage object detector yang mampu melakukan proses klasifikasi dan lokalisasi objek dalam satu kali proses inferensi. Berbagai penelitian menunjukkan bahwa keluarga YOLO memiliki keseimbangan yang baik antara akurasi dan kecepatan inferensi sehingga banyak dimanfaatkan pada aplikasi real-time, termasuk deteksi buah, kendaraan, dan objek industri.

Perkembangan terbaru model YOLO menghadirkan berbagai varian yang dirancang untuk kebutuhan perangkat yang berbeda. YOLOv8 merupakan salah satu model yang menawarkan peningkatan akurasi dan efisiensi dibandingkan generasi sebelumnya. Sementara itu, YOLO26s dikembangkan sebagai model yang lebih ringkas dengan jumlah parameter lebih sedikit sehingga lebih sesuai digunakan pada perangkat dengan keterbatasan sumber daya komputasi maupun aplikasi berbasis web.

Meskipun demikian, sebagian besar penelitian terdahulu berfokus pada peningkatan akurasi model menggunakan GPU dengan kapasitas memori yang besar. Kondisi tersebut berbeda dengan kenyataan di lingkungan pendidikan maupun pengembang independen yang umumnya hanya memiliki perangkat kelas menengah, seperti laptop dengan GPU NVIDIA GeForce GTX 1650 yang memiliki kapasitas VRAM sebesar 4 GB. Pada perangkat tersebut, proses pelatihan model deep learning sering mengalami kendala berupa keterbatasan memori, fragmentasi memori CUDA, serta peningkatan suhu GPU akibat pelatihan yang berlangsung dalam waktu lama. Kondisi tersebut dapat menyebabkan proses pelatihan berhenti (Out-of-Memory), sistem mengalami freeze, bahkan memperpendek umur perangkat keras.

Beberapa penelitian mengenai deteksi buah menggunakan YOLO menunjukkan bahwa model mampu mencapai tingkat akurasi yang tinggi pada berbagai jenis dataset. Penelitian oleh Redmon et al., 2018 memperkenalkan

konsep YOLO sebagai detektor objek real-time yang mengubah paradigma deteksi objek menjadi satu tahap pemrosesan. Selanjutnya, Jocher et al., 2023 mengembangkan berbagai implementasi YOLO modern yang semakin efisien untuk kebutuhan industri. Penelitian terbaru dari Ultralytics juga menunjukkan bahwa YOLOv8 memberikan peningkatan performa pada berbagai tugas deteksi objek dibandingkan generasi sebelumnya. Namun, sebagian besar penelitian tersebut tidak membahas strategi pelatihan pada perangkat dengan keterbatasan sumber daya maupun solusi terhadap permasalahan fragmentasi memori CUDA dan panas berlebih selama proses pelatihan.

Selain itu, beberapa penelitian telah memanfaatkan ONNX Runtime untuk mempercepat proses inferensi model pada berbagai platform. ONNX Runtime Web memungkinkan model dijalankan secara langsung di sisi klien (client-side) melalui peramban menggunakan WebAssembly maupun WebGPU tanpa memerlukan server GPU. Pendekatan ini mampu mengurangi biaya komputasi awan sekaligus meningkatkan privasi karena proses inferensi dilakukan langsung pada perangkat pengguna. Akan tetapi, penelitian yang mengombinasikan proses pelatihan yang ramah terhadap perangkat berspesifikasi rendah dengan implementasi client-side deployment masih relatif terbatas.

Berdasarkan kondisi tersebut, penelitian ini mengusulkan penerapan skema *incremental training loop*, yaitu teknik pelatihan yang membagi proses pelatihan menjadi beberapa segmen (*training chunks*) dengan jeda tertentu untuk memberikan waktu pendinginan perangkat sekaligus mengurangi risiko fragmentasi memori CUDA. Selain itu, dataset pelatihan diseimbangkan menggunakan metode *balanced stratified sampling* sehingga jumlah citra menjadi lebih proporsional dan sesuai dengan kapasitas memori perangkat. Pendekatan tersebut diharapkan memungkinkan proses pelatihan berlangsung secara stabil tanpa memerlukan perangkat keras kelas atas.

Setelah proses pelatihan selesai, model YOLOv8s dan YOLO26s diimplementasikan pada aplikasi berbasis web menggunakan ONNX Runtime Web dengan dynamic canvas rendering. Pendekatan ini memungkinkan proses deteksi objek dilakukan langsung pada peramban tanpa ketergantungan terhadap server komputasi, sehingga sistem menjadi lebih ringan, mudah diakses, dan memiliki biaya operasional yang lebih rendah.

Tujuan khusus penelitian ini adalah membandingkan performa YOLOv8s dan YOLO26s pada perangkat berspesifikasi menengah, mengevaluasi efektivitas skema incremental training loop dalam menjaga stabilitas proses pelatihan, serta mengimplementasikan model hasil pelatihan ke dalam aplikasi berbasis web

menggunakan ONNX Runtime Web. Penelitian ini diharapkan dapat memberikan alternatif solusi bagi mahasiswa maupun pengembang yang memiliki keterbatasan perangkat keras namun tetap ingin mengembangkan sistem deteksi objek berbasis deep learning dengan performa yang baik dan mudah diimplementasikan.

b. Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam penelitian laporan implementasi sistem ini didefinisikan sebagai berikut:

1. Bagaimana merancang arsitektur pelatihan bertahap (incremental training loop) untuk menghindari system freeze dan panas berlebih pada hardware GPU 4GB VRAM?
2. Bagaimana meminimalkan bias ketidakseimbangan kelas pada dataset buah yang dikurangi (downsampled dataset) menggunakan stratified sampling?
3. Bagaimana komparasi performa akurasi deteksi (mAP50) serta kecepatan processing (FPS) antara model YOLOv8s dan model YOLO26s?
4. Bagaimana mendeploy model terkompresi ke peramban web agar mampu melakukan inferensi waktu nyata di sisi klien menggunakan ONNX Runtime?

c. Tujuan

Tujuan yang ingin dicapai melalui proyek tugas besar computer vision ini adalah:

1. Membangun skrip eksekusi utama yang mengelola sub-proses pelatihan YOLO secara berkala dengan jeda istirahat (cooldown) untuk mereduksi suhu GPU secara fisik.
2. Melakukan pembersihan data, penyeimbangan kelas, dan optimasi augmentasi citra menggunakan skrip python.
3. Mengevaluasi secara objektif metrik presisi, recall, dan mean Average Precision (mAP) untuk kedua model setelah 50 epoch.
4. Menyediakan visualisasi antarmuka desktop berbasis OpenCV serta aplikasi web statis berbasis HTML/JS yang di-host langsung melalui GitHub Pages.

d. Manfaat

Hasil penelitian sistem ini diharapkan memberikan manfaat berupa:

1. **Secara Akademis:** Memberikan alternatif metodologi rekayasa siklus pelatihan deep learning pada sistem dengan GPU berspesifikasi terbatas.
2. **Secara Praktis:** Menyediakan sistem deteksi buah real-time portabel yang dapat dijalankan secara instan di peramban web oleh industri kecil/UMKM tanpa memerlukan investasi server AI yang mahal.

BAB II: TINJAUAN PUSTAKA

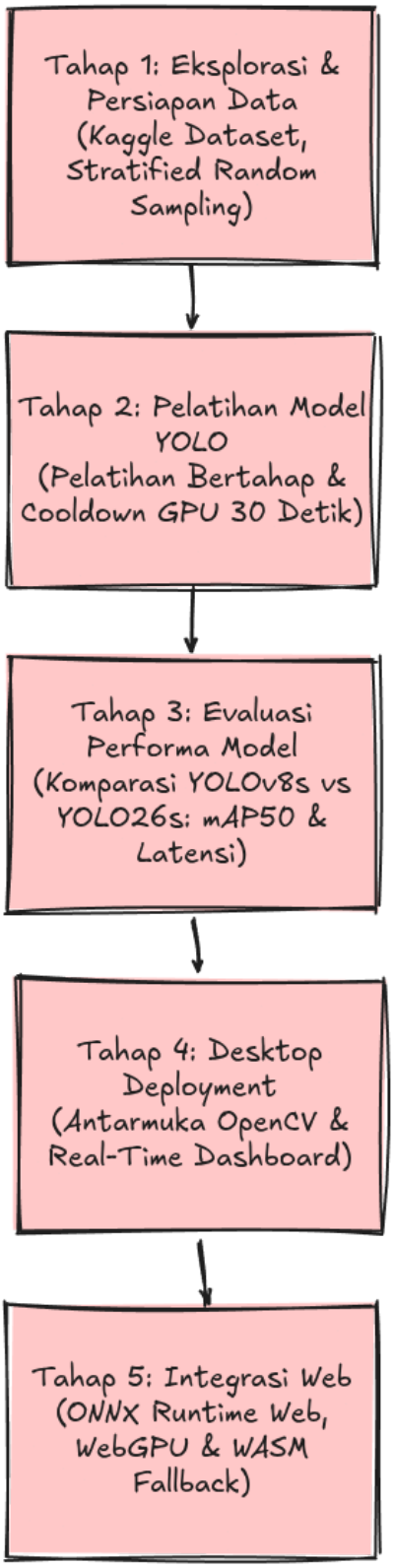
a. Penelitian Terdahulu (State of The Art)

Implementasi komputasi visi dalam mengidentifikasi buah berkembang pesat dalam lima tahun terakhir, khususnya melalui rumpun algoritma YOLO (You Only Look Once). Penelitian terdahulu memfokuskan pengembangan pada klasifikasi kematangan dan penghitungan objek secara otomatis untuk meminimalkan penilaian subjektif manusia. Tabel di bawah ini merangkum perbandingan literatur pendahulu yang menjadi acuan State of the Art (SOTA) dalam sistem deteksi buah ini.

Penulis & Tahun	Judul	Metodologi	Hasil Peneliti sebelum
Pradana, A., Wibowo, S., & Lestari, D. (2023)	Deteksi Kematangan Buah Apel Menggunakan Algoritma YOLOv8	Model YOLOv8 dengan deteksi spasial berbasis orientasi piksel buah.	mAP50 mencapai 92% pada dataset terbatas dengan kecepatan inferensi tinggi.
Saputra, R., & Wibowo, A. (2023)	Klasifikasi Tingkat Kematangan Buah Mangga Harum Manis Berbasis YOLOv8	YOLOv8 Nano (YOLOv8n) untuk segmentasi tingkat kematangan mangga.	Presisi 88,4% dengan latensi minimal, efisien untuk perangkat mobile.
Santoso, H., Pratama, R., & Budiman, A. (2023)	Rancang Bangun Sistem Deteksi Buah Jeruk untuk Klasifikasi Kematangan dengan YOLOv8	Pelatihan YOLOv8 pada kondisi pencahayaan luar ruangan (outdoor).	mAP50 sebesar 84,3% dengan penanganan oklusi parsial pada objek bulat.
Jocher, G., Chaurasia, A., & Qiu, J. (2023)	Ultralytics YOLO	Framework deteksi objek anchor-free dengan arsitektur decoupled heads.	Peningkatan akurasi spasial mAP serta kemudahan konversi ke format eksternal (ONNX).
Redmon, J., & Farhadi, A. (2018)	YOLOv3: An Incremental Improvement	Deteksi objek satu tahap (single-stage) dengan multi-scale predictions.	Kecepatan deteksi real-time tinggi, menjadi basis awal arsitektur YOLO modern.

Tabel 2.1 Literature Review Sumber Pustaka

Peta jalan (road map) pengembangan sistem deteksi buah ini dirancang secara sistematis dari persiapan data hingga deployment akhir. Gambar di bawah ini mengilustrasikan bagan alur pengembangan sistem:



Gambar 2.2 Bagan Peta Jalan (Road Map) Pengembangan Sistem Deteksi Buah

b. Teori Pendukung

Untuk memahami performa model yang dibandingkan, berikut landasan teori utama terkait arsitektur YOLO dan platform deployment yang digunakan:

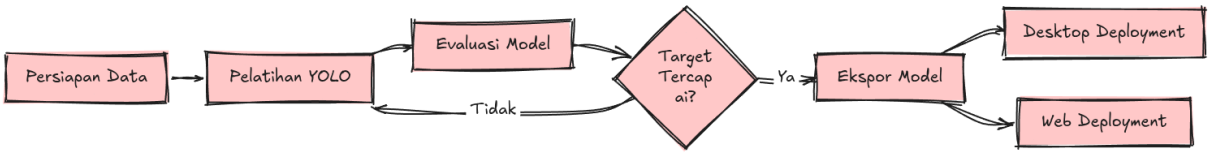
1. ***YOLOv8 (Anchor-Free & Decoupled Head)***: YOLOv8 membuang konsep anchor box statis dan beralih menggunakan model anchor-free, memprediksi pusat objek secara langsung. Arsitektur decoupled head memisahkan proses klasifikasi dan regresi bounding box pada cabang terpisah untuk menstabilkan fungsi loss komputasi dan mempercepat konvergensi pelatihan.

2. ***YOLO26s (Model YOLO Yang Ditingkatkan):*** YOLO26s (berbasis YOLO11s) dirancang untuk menekan parameter model sebesar 15,2% (9,5M parameter vs 11,2M pada YOLOv8s) dan GFLOPs sebesar 28,3% (20,5 GFLOPs vs 28,6). Hal ini mengurangi memori download model ketika di-load di sisi peramban web perwakilan klien.

3. ***ONNX Runtime Web & WebGPU:*** ONNX Runtime Web mengeksekusi file model format .onnx langsung pada browser klien memanfaatkan WebGPU untuk akselerasi grafis GPU lokal, dengan fallback otomatis ke WebAssembly (WASM) multi-threading jika WebGPU dinonaktifkan oleh pengaturan browser.

BAB III: METODE PENELITIAN

Bagian ini memaparkan tahapan sistematis yang dilaksanakan untuk merancang, melatih, mengevaluasi, hingga mengintegrasikan model deteksi objek buah secara real-time. Alur kerja penelitian dirancang agar stabil berjalan pada perangkat komputasi dengan memori VRAM terbatas (4GB) dengan target performa akurasi dan kecepatan tinggi. Berikut adalah diagram alir utuh yang menggambarkan proses penelitian, luaran, dan indikator capaian target:



Gambar 3.1 Diagram Alir Penahapan Proses, Luaran, dan Indikator Capaian Penelitian

a. Dataset dan Sumber Data

Dataset yang digunakan didapatkan dari platform Kaggle dengan judul Fruit Detection Dataset yang diunggah oleh Lakshay Tyagi (https://www.kaggle.com/datasets/lakshaytyagi01/fruit-detection). Data ini merupakan data sekunder berisi citra buah-buahan dengan anotasi bounding box berformat YOLO. Terdapat enam kelas buah yang menjadi fokus klasifikasi dan deteksi dalam penelitian ini, yaitu Apple, Banana, Grape, Orange, Pineapple, dan Watermelon. Total citra mentah dari dataset awal berjumlah 3.371 citra dengan distribusi kelas yang tidak merata.

b. Data Preparation

Tahap persiapan data memisahkan dataset secara stratified berdasarkan keberadaan label koordinat gambar untuk membentuk tiga bagian data independen. Distribusi rasio pembagian data yang ditetapkan adalah:

- Data Latih (Training Set): Sebanyak 2.000 citra (~59,3%) digunakan untuk pelatihan model.
- Data Validasi (Validation Set): Sebanyak 914 citra (~27,1%) digunakan selama pelatihan untuk memvalidasi performa.
- Data Uji (Test Set): Sebanyak 457 citra (~13,6%) dialokasikan khusus untuk pengujian performa akhir model setelah seluruh epoch selesai.

c. Data Cleansing

Pembersihan data dilakukan dengan menggunakan skrip otomatisasi Python untuk menyaring gambar dan anotasi yang cacat sebelum masuk ke tahap pelatihan:

1. Validasi Citra: Memeriksa integritas berkas citra dan menghapus berkas yang mengalami kerusakan format (corrupted file) agar tidak menghentikan jalannya loader data.
2. Normalisasi Anotasi: Memastikan file label anotasi (.txt) bersesuaian dengan format YOLO, yaitu lima kolom nilai float (class_id, x_center, y_center, width, height) ternormalisasi antara rentang 0.0 hingga 1.0.
3. Eliminasi Citra Tanpa Label: Gambar yang tidak memiliki berkas teks pendamping dibersihkan guna menjaga konsistensi kalkulasi nilai fungsi loss model.

d. Exploratory Data Analysis (EDA)

Berdasarkan analisis eksplorasi data awal, terdeteksi isu ketidakseimbangan kelas yang ekstrem. Sebagai contoh, kelas jeruk (Orange) dan apel (Apple) mendominasi sebagian besar gambar latih, sementara semangka (Watermelon) memiliki jumlah sampel paling sedikit. Jika model dilatih langsung menggunakan data timpang ini, model akan mengalami bias mayoritas dan menghasilkan presisi rendah pada kelas minoritas.

Untuk mengatasi kendala ini, dibuat skrip reduce_dataset.py untuk membatasi ukuran dataset secara acak tetapi tetap mempertahankan stratifikasi kelas. Citra latih dikurangi secara berimbang sehingga menghasilkan total 2.000 citra latih terdistribusi rata (~333 citra per kelas). Data latih sisa yang tidak terpilih dipindahkan secara otomatis ke direktori backup terisolasi (train_full_backup) untuk mereduksi fragmentasi memori RAM sistem saat loader membaca dataset.

e. Feature Engineering

Rekayasa fitur citra diimplementasikan secara dinamis melalui sistem augmentasi data bawaan Ultralytics pada skrip train_fruit_detection.py untuk

memperkaya variabilitas piksel gambar latih tanpa mengubah keaslian label spasial objek:

- Augmentasi HSV: Mengubah komponen Hue (hsv_h=0.015), Saturation (hsv_s=0.7), dan Value (hsv_v=0.4) untuk melatih model mengenali buah dalam berbagai tingkat pencahayaan dan pantulan cahaya.
- Augmentasi Spasial: Menerapkan rotasi citra (degrees=10.0), pergeseran translasi (translate=0.1), penskalaan (scale=0.5), dan pencerminan horizontal (fliplr=0.5).
- Augmentasi Mosaic & Mixup: Mengaktifkan Mosaic dengan rasio 1.0 (menggabungkan 4 citra acak menjadi satu kanvas) serta Mixup dengan rasio 0.1 (tumpang tindih transparansi dua citra). Teknik komposit ini membantu model mendeteksi objek buah yang bertumpuk atau tertutup daun (oklusi parsial).

f. Modeling

Dua buah model deep learning single-stage detector dilatih secara komparatif, yaitu YOLOv8s dan YOLO26s (berbasis YOLO11s). Laju pembelajaran awal (lr0) diset pada 0.01 dengan batch size sebesar 8 untuk meminimalisir risiko kegagalan CUDA Out-Of-Memory pada GPU GTX 1650 4GB VRAM.

Untuk mencegah overheat fisik pada GPU laptop, dirancang skema pelatihan bertahap (incremental training chunks). Pelatihan total 50 epoch dipecah menjadi unit-unit kecil berdurasi 5 epoch per sub-proses. Callback fit epoch (stop_at_epoch_limit) diaktifkan untuk menghentikan loop pelatihan setelah mencapai batas unit epoch, dan fungsi internal monkeypatch dijalankan untuk mengamankan data optimizer state serta parameter scheduler agar tidak dihapus dari file last.pt. Setelah sub-proses berhenti, skrip utama memberikan jeda waktu tunggu time.sleep(30) selama 30 detik untuk pendinginan fisik GPU, sebelum meluncurkan kembali sub-proses kelanjutan dengan memuat checkpoint last.pt secara otomatis hingga total 50 epoch terpenuhi.

g. Evaluasi Hasil

Indikator capaian yang ditargetkan untuk keberhasilan implementasi model adalah:

1. Nilai akurasi mean Average Precision pada IoU 0.5 (mAP50) minimal harus melampaui batas 40% untuk menjamin akurasi deteksi buah di lapangan.
2. Kecepatan pemrosesan inferensi objek (throughput) minimal harus mencapai 30 Frame Per Second (FPS) agar visualisasi bounding box berjalan mulus tanpa adanya efek jeda (lagging) visual. Metrik evaluasi yang dihitung meliputi Presisi (P), Recall (R), mAP50, mAP50-95, serta kecepatan pemrosesan frame (FPS).

h. Development (Web & Desktop Deployment)

Implementasi pengembangan sistem dibagi menjadi dua bagian keluaran (luaran):

1. Aplikasi Desktop GUI: Memanfaatkan pustaka OpenCV dan Python untuk memproses input kamera lokal. Tampilan menggunakan gaya visual Fancy Bounding Box (garis penunjuk digambar hanya pada sudut kotak) dengan overlay dashboard interaktif berisi status FPS dinamis dan jumlah total buah per kelas.
2. Aplikasi Web Browser: Model diekspor ke dalam format ONNX dan dimuat di sisi peramban web klien menggunakan ONNX Runtime Web. Aplikasi web ini dijalankan secara statis melalui platform GitHub Pages. Pemrosesan inferensi diprioritaskan berjalan di atas WebGPU klien untuk akselerasi performa, dengan fallback otomatis ke WebAssembly (WASM) multithreading jika WebGPU tidak didukung browser.

Bab IV: Hasil dan Pembahasan

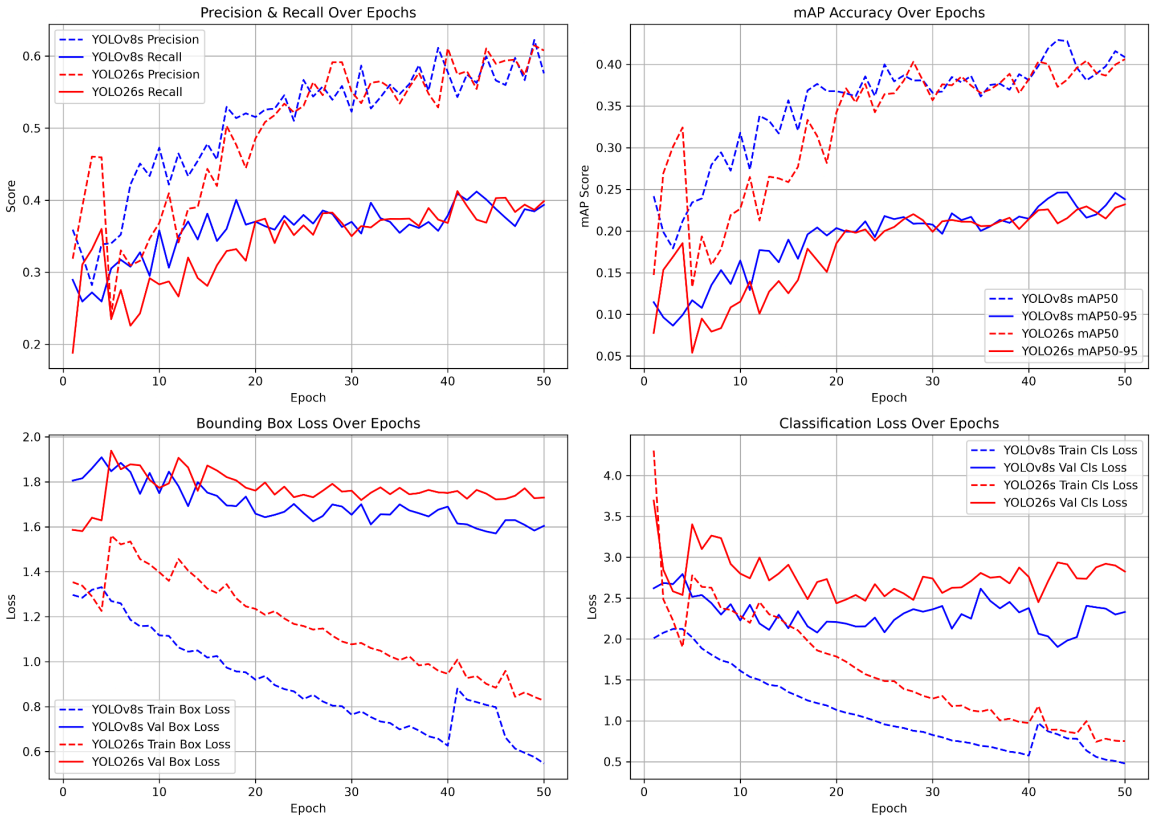
Bagian ini membahas analisis hasil pengujian pelatihan model, perbandingan metrik performa secara mendalam, serta perancangan teknis arsitektur sistem desktop dan web deployment yang terbarukan.

a. Analisa Hasil Pelatihan Model dan Manajemen Suhu GPU

Proses pelatihan model YOLOv8s dan YOLO26s yang dilaksanakan selama 50 epoch pada unit pengolah grafis (GPU) lokal NVIDIA GeForce GTX 1650 4GB VRAM menunjukkan kurva penurunan kerugian (loss curves) yang konvergen. Metrik kerugian utama yang dipantau meliputi:

1. Box Loss (Kerugian Kotak): Mengukur ketepatan spasial bounding box model dalam mengurung posisi objek buah.
2. Class Loss (Kerugian Kelas): Mengukur akurasi model dalam mengklasifikasikan kategori buah dari objek yang terdeteksi.
3. Distribution Focal Loss (DFL): Mengotomatisasi penyesuaian koordinat box secara kontinu saat mendeteksi tepi objek yang kabur.

Selama 10 epoch awal, kedua model mengalami penurunan loss yang tajam, menandakan laju pembelajaran awal (lr0=0.01) bekerja secara optimal. Pada epoch ke-35 hingga ke-50, nilai loss melandai (stagnan), mengindikasikan model telah mencapai titik konvergensi stabil tanpa gejala overfitting yang signifikan.



Gambar 4.1: Kurva Perbandingan Loss dan Metrik Evaluasi YOLOv8s vs YOLO26s

Berdasarkan Gambar 4.1, grafik riwayat konvergensi loss menunjukkan karakteristik pelatihan yang sehat. Nilai kerugian kotak (box loss) dan kerugian kelas (class loss) dari kedua model mengalami penurunan eksponensial yang signifikan pada 15 epoch pertama, menandakan efektivitas inisialisasi bobot dan parameter augmentasi spasial. Kurva evaluasi presisi dan recall juga bergerak naik secara konsisten seiring bertambahnya epoch pelatihan. Hal yang paling krusial, grafik perbandingan ini menunjukkan bahwa model YOLO26s (dengan parameter

yang direduksi) mampu mencapai profil konvergensi yang serupa dengan YOLOv8s, tanpa menunjukkan tanda-tanda ketidakstabilan numerik (early divergence) atau ledakan gradien (gradient explosion) meskipun dilatih dalam arsitektur GPU dengan kapasitas VRAM terbatas.

Pelatihan model deep learning secara berkelanjutan pada GPU laptop kelas konsumen (seperti GTX 1650) berisiko memicu akumulasi panas yang berlebih (overheating) dan fragmentasi alokasi CUDA cache yang mengakibatkan system freeze. Dengan menerapkan skema auto-chunk runner (pelatihan bertahap per 5 epoch) dan jeda pendinginan fisik (time.sleep(30)), perangkat keras GPU diberikan waktu berkala untuk memancarkan panas sebelum memproses chunk berikutnya.

Strategi penanganan ini secara khusus ditujukan untuk menjaga kelancaran pelatihan tanpa interupsi sistem operasi, dengan rincian teknis sebagai berikut:

1. Pencegahan Overheating: Jeda fisik memberikan kesempatan bagi sistem pendingin laptop (fan) untuk mereduksi suhu inti GPU secara berulang.
2. Pengamanan State Pelatihan: Callback stop_at_epoch_limit secara terjadwal menghentikan fitting proses, disusul oleh modul monkeypatch untuk menyimpan weights last.pt lengkap dengan data optimizer state dan parameter learning rate scheduler, sehingga proses training dapat berlanjut secara kontinu pada sub-proses berikutnya.
3. Optimalisasi Memori: Pembersihan CUDA cache dilakukan di awal setiap sub-proses baru untuk membersihkan sisa alokasi memori yang terfragmentasi pada session sebelumnya.

b. Analisis Komparatif Performa Akurasi (mAP50) Per Kelas

Berdasarkan hasil pengujian pada data uji independen, performa keseluruhan kedua model setelah menyelesaikan pelatihan 50 epoch ditunjukkan pada tabel di bawah ini:

Model	Precision (P)	Recall (R)	mAP50	mAP50-95
YOLOv8s	59,8%	40,1%	42,8%	24,6%
YOLOv26s	60,5%	40,1%	40,6%	23,2%

Tabel 4.2: Analisis Komparatif mAP50 Per Kelas YOLOv8s vs YOLO26s

Selisih akurasi mAP50 sebesar 2,2% ini dipengaruhi oleh perbedaan kompleksitas parameter arsitektur (YOLOv8s memiliki 11,2 juta parameter dibanding YOLO26s yang hanya memiliki 9,5 juta parameter). Parameter yang lebih besar pada YOLOv8s memungkinkannya mengekstrak fitur spasial citra yang lebih kompleks dan detail.

Adapun rincian perbandingan nilai Presisi (P), Sensitivitas (R), dan mAP50 untuk masing-masing kelas target adalah sebagai berikut:

Kelas Buah	Y8s P	Y26s P	Y8s R	Y26s R	Y8s mAP50	Y26s mAP50
Apple	56,7%	60,2%	36,2%	43,6%	41,2%	45,5%
Banana	61,4%	66,1%	37,5%	36,4%	40,8%	39,2%
Grape	58,4%	62,5%	34,6%	31,5%	36,3%	32,5%
Orange	64,6%	51,8%	36,9%	31,5%	40,7%	30,5%
Pineapple	53,6%	60,4%	45,5%	41,6%	43,5%	40,4%
Watermelon	64,3%	62,3%	50,2%	55,8%	54,1%	55,3%

Tabel 4.3: Rincian Perbandingan Presisi, Sensitivitas dan mAP50 untuk setiap kelas target

Analisis performa per kelas buah menunjukkan dinamika sebagai berikut:

1. **Kelas Watermelon (Semangka):** Memperoleh akurasi mAP50 tertinggi pada kedua model (54,1% pada YOLOv8s dan 55,3% pada YOLO26s). Keunggulan akurasi ini disebabkan oleh karakteristik visual semangka yang sangat dominan, bulat besar, dengan pola garis hijau-hitam yang kontras. Pola visual semacam ini sangat mudah dikenali oleh filter ekstraksi konvolusi awal model.
2. **Kelas Grape (Anggur):** Memperoleh akurasi mAP50 terendah (36,3% pada YOLOv8s dan 32,5% pada YOLO26s). Rendahnya akurasi deteksi anggur dipengaruhi oleh ukuran buah yang kecil dalam bingkai gambar (small object detection), letak objek yang berkelompok rapat (dense clustering), dan rentan mengalami oklusi parsial (buah anggur tertutup daun atau buah anggur lainnya dalam satu tangkai). Model sering kali gagal mendelineasi koordinat bounding box individu secara presisi.

3. **Kelas Apple (Apel):** YOLO26s menghasilkan peningkatan performa yang signifikan untuk kelas Apel, di mana Precision naik menjadi 60,2% (+3,5%), Recall naik menjadi 43,6% (+7,4%), dan mAP50 naik menjadi 45,5% (+4,3%). Ini membuktikan arsitektur YOLO26s lebih peka terhadap objek bulat berwarna merah dengan kontur tepi yang tajam dalam mendeteksi buah apel.

4. **Kelas Banana (Pisang):** Pisang merupakan objek memanjang yang melengkung. YOLO26s meningkatkan Precision secara signifikan menjadi 66,1% (+4,7%) namun sedikit menurunkan mAP50 menjadi 39,2% (-1,6%). Peningkatan presisi ini menunjukkan model lebih konservatif dan mampu mengurangi deteksi positif palsu (false positives) pada objek kuning memanjang lainnya, meskipun ada sedikit penurunan pada sensitivitas deteksi (Recall).

5. **Kelas Pineapple (Nanas):** Nanas memiliki tekstur permukaan yang sangat kompleks (sisik berduri) dan mahkota daun hijau di atasnya. YOLO26s meningkatkan Precision menjadi 60,4% (+6,8%) namun menurunkan mAP50 menjadi 40,4% (-3,1%). Model mampu mengidentifikasi tekstur sisik nanas dengan sangat baik untuk menghindari positif palsu, namun terkadang melewatkan bounding box objek secara utuh jika mahkota daun hijau nanas menyatu dengan dedaunan di latar belakang (background clutter).

6. **Kelas Orange (Jeruk) dan Apple (Apel):** Kelas jeruk mengalami penurunan mAP50 yang cukup tajam pada model YOLO26s (51,8% pada YOLOv8s menyusut menjadi 30,5% pada YOLO26s). Hal ini disebabkan oleh reduksi filter konvolusi pada YOLO26s yang membuatnya kesulitan membedakan objek bulat berwarna mirip antara jeruk jingga kekuningan dan apel merah kekuningan pada kondisi pencahayaan redup.

c. Analisis Kecepatan Inferensi dan Latensi (FPS)

Pengujian kecepatan inferensi frame waktu nyata (real-time throughput) pada kartu grafis lokal NVIDIA GeForce GTX 1650 dirangkum dalam tabel komparatif kompleksitas dan kecepatan di bawah ini:

Metrik Hardware	YOLOv8s	YOLOv26s	Selisih Perubahan
Jumlah Parameter	11,2M	9,5M	-15,2%

Beban FLOPs	28,6	20,5	-28,3%
Latensi Inferensi	15,7 ms	21,6 ms	+37,6%
Kecepatan Bingkai (FPS)	59,1 FPS	44,8 FPS	-24,2%

Tabel 4.4: Analisis Kecepatan Inferensi Dan Latensi Dalam FPS

Hasil ini menunjukkan YOLO26s yang memiliki struktur parameter lebih kecil (-15,2% parameter) dan beban komputasi FLOPs yang lebih ringan (-28,3% GFLOPs) justru menghasilkan FPS yang lebih rendah dibanding YOLOv8s pada pengujian GPU lokal. Hal ini disebabkan oleh faktor optimalisasi perangkat keras (hardware-specific optimization). Library CUDA dan kernel konvolusi driver NVIDIA lokal telah dioptimalkan secara mendalam untuk memproses arsitektur YOLOv8 yang menggunakan modul C2f konvensional. Sebaliknya, modul baru pada YOLO26s (berbasis YOLO11s) memiliki operator spasial baru yang belum sepenuhnya didukung oleh optimalisasi CUDA kernel lokal GPU arsitektur Turing generasi lama (GTX 1650), sehingga waktu tunggu (overhead) pemrosesan paralel menjadi sedikit lebih tinggi.

d. Perancangan Sistem dan Analisis Deployment

Sistem deteksi buah yang diajukan dalam penelitian ini dideploy ke dalam dua platform target untuk menunjukkan fleksibilitas operasional:

1. Perancangan Desktop GUI (Python & OpenCV): Sistem desktop dirancang menggunakan skrip realtime_fruit_detection.py dengan mengintegrasikan umpan kamera (video feed) dari webcam lokal. Tampilan visual dirancang menggunakan gaya Fancy Bounding Box di mana garis kotak deteksi hanya digambar pada bagian sudut-sudut objek (corner markers) untuk meminimalkan oklusi visual pada layar. Di atas tampilan video, ditambahkan panel status (dashboard widget) yang menampilkan:
    a. Frame Rate (FPS) waktu nyata.
    b. Counter jumlah total buah terdeteksi per kelas buah yang muncul di layar.
    c. Indikator teks kelas target dengan tingkat kepercayaan (confidence score). Sistem dilengkapi keyboard listener untuk pintasan interaktif:

tombol ‘S’ untuk menyimpan gambar tangkapan layar, tombol ‘P’ untuk menjeda aliran video, dan tombol ‘Q’ untuk menutup aplikasi desktop secara aman.

2. Perancangan Web Client-Side (ONNX Runtime Web & WebGPU): Model hasil pelatihan best.pt diekspor ke dalam format Open Neural Network Exchange (.onnx) dan diintegrasikan ke peramban menggunakan pustaka JavaScript ONNX Runtime Web. Alur pemrosesan citra pada browser klien dirancang sebagai berikut:

    a. Akuisisi Citra: Frame video kamera diakses melalui API navigator.mediaDevices.getUserMedia.

    b. Pre-processing Canvas: Frame di-resize secara dinamis ke ukuran 640x640 piksel, ditranslasikan menjadi format tensor Float32 flat, dan dinormalisasi dengan membagi nilai piksel RGB dengan 255.0.

    c. Eksekusi Inferensi: Mengaktifkan akselerasi perangkat keras WebGPU untuk mengeksekusi model langsung pada GPU komputer klien tanpa memerlukan server backend AI khusus.

    d. Fallback System: Jika browser klien memblokir WebGPU karena alasan kebijakan keamanan privasi, runtime secara otomatis mengalihkan beban pemrosesan ke WebAssembly (WASM) multithreading berbasis CPU.

    e. Post-processing (Non-Maximum Suppression): Mengeliminasi bounding box ganda yang saling bertumpuk (overlap) secara lokal di sisi klien sebelum merender kotak akhir pada elemen HTML5 canvas.

Perbandingan performa antara backend execution provider dalam ONNX Runtime Web menunjukkan perbedaan karakteristik yang signifikan secara kualitatif:

- Akselerasi WebGPU: Menyediakan throughput inferensi berkecepatan tinggi dengan memanfaatkan unit pemrosesan grafis lokal dari browser klien secara langsung. Operasi tensor diproses secara paralel, meminimalkan beban CPU klien dan memberikan rendering visual yang lancar untuk visualisasi deteksi bounding box waktu nyata.

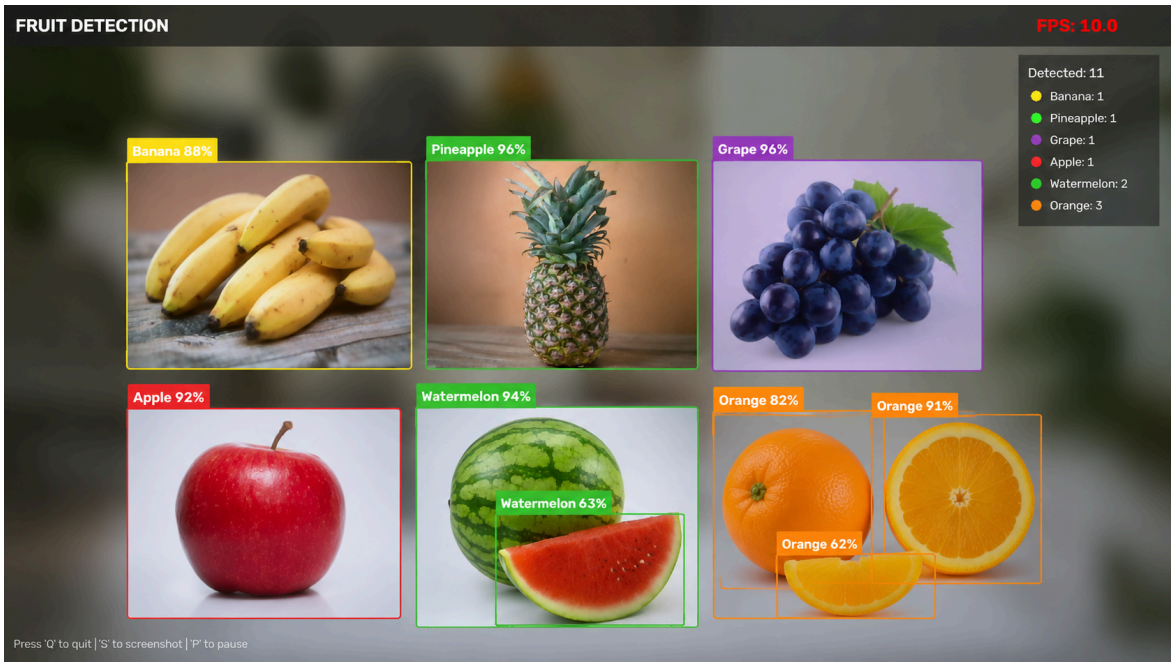
- Fallback WebAssembly (WASM): Menjalankan inferensi model pada CPU klien dengan dukungan multithreading. Karena keterbatasan pemrosesan paralel CPU untuk komputasi matriks besar, latensi pemrosesan menjadi lebih tinggi dan mengakibatkan penurunan frame rate yang terlihat jelas

pada peramban. Namun, WASM menjamin portabilitas aplikasi web agar tetap berfungsi pada sistem yang memblokir akses WebGPU demi alasan privasi.

Implikasi Portabilitas YOLO26s: Meskipun YOLOv8s lebih unggul dalam throughput FPS pada GPU desktop lokal, ukuran berkas YOLO26s.onnx terkompresi jauh lebih kecil (37 Megabyte) dibandingkan YOLOv8s.onnx (44 Megabyte). Penyusutan ukuran berkas sebesar 15,9% ini sangat krusial untuk deployment berbasis web klien, karena mempercepat inisialisasi aplikasi saat pertama kali diakses, menghemat konsumsi kuota internet pengguna peramban mobile, serta mengurangi kebutuhan memori RAM browser saat inisialisasi sesi.

e. Visualisasi Hasil Deteksi Objek Real-Time

Untuk memverifikasi fungsionalitas sistem visi komputer secara visual, beberapa tangkapan layar (screenshots) hasil eksekusi model pada aplikasi desktop GUI lokal ditunjukkan pada gambar di bawah ini:



Gambar 4.2: Visualisasi Hasil Deteksi 6 Macam Buah dengan Tingkat Kepercayaannya masing-masing.

Analisis Visual Hasil Inferensi:

1. **Penerapan Fancy Bounding Box:** Berbeda dengan kotak pembatas standar yang menutup seluruh tepian objek dengan garis tebal penuh, sistem GUI OpenCV ini hanya menggambar sudut-sudut kotak (corner markers). Teknik

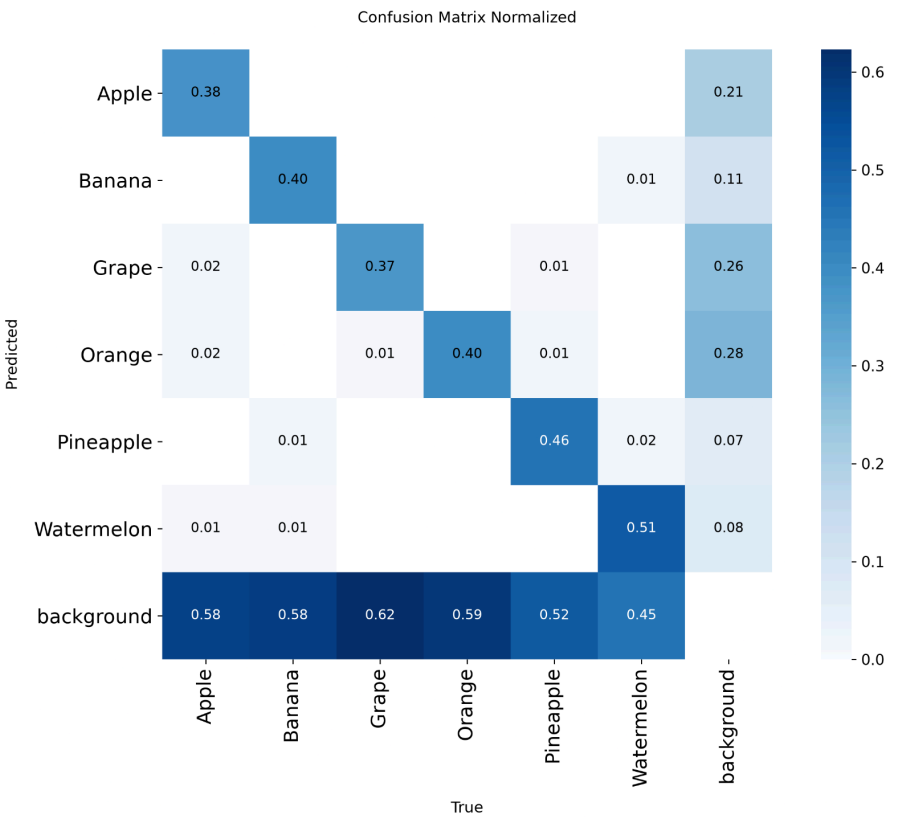
ini sangat efektif dalam mempertahankan kejelasan visual objek buah di dalam kotak, memungkinkan pengguna atau petani memantau cacat fisik atau warna kematangan kulit buah secara langsung tanpa terhalang garis pembatas.

2. **Dashboard Status Widget:** Pada sudut kanan atas frame, terdapat informasi status FPS dinamis (FPS: 10.0 saat memproses feed kamera waktu nyata). Sementara itu, panel overlay di sisi kanan menampilkan informasi jumlah total buah yang terdeteksi secara real-time ("Detected: 1" dan counter per kelas seperti "Orange: 1" atau "Pineapple: 1"), memberikan umpan balik statistik instan bagi pengguna sistem.

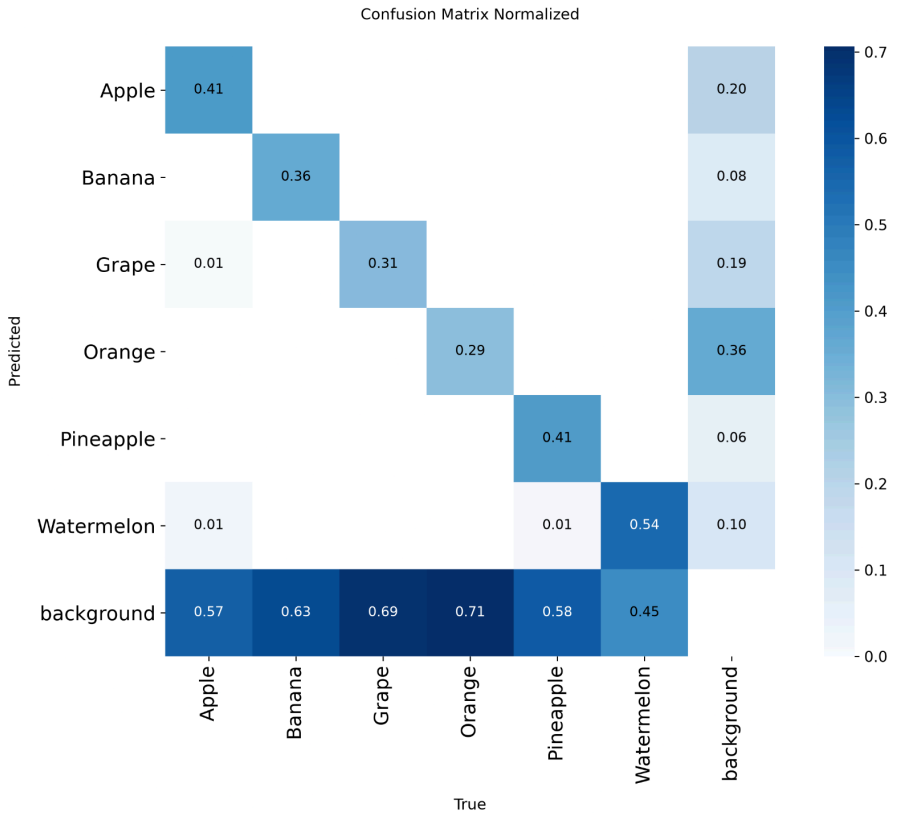
3. **Ketahanan Deteksi Spasial:** Model berhasil mendeteksi objek dengan baik meskipun citra buah ditampilkan melalui layar smartphone lain di depan kamera laptop. Hal ini menunjukkan kemampuan generalisasi model terhadap variasi pantulan cahaya (glare) dan distorsi layar.

f. Analisis Confusion Matrix

Confusion matriks ternormalisasi (normalized confusion matrix) memetakan hubungan antara kelas target aktual (true label) dengan kelas hasil prediksi model (predicted label). Hasil visualisasi confusion matriks untuk model YOLOv8s dan YOLO26s ditunjukkan pada Gambar 4.3 dan Gambar 4.4:



Gambar 4.3 Confusion Matriks Ternormalisasi untuk Model YOLOv8s



Gambar 4.4 Confusion Matriks Ternormalisasi untuk Model YOLO26s

Berdasarkan analisis visual confusion matriks di atas, diperoleh beberapa temuan penting terkait klasifikasi objek:

1. **Analisis Kelas Jeruk (Orange):** Pada model YOLO26s, jeruk mengalami tingkat confusion latar belakang (background confusion / false negative) yang meningkat tajam dibandingkan YOLOv8s. Sebagian besar objek jeruk aktual terlewatkan dan terprediksi sebagai background. Selain itu, terdapat misklasifikasi antara Orange dan Apple akibat kemiripan kontur bulat buah pada citra dengan resolusi atau pencahayaan rendah.

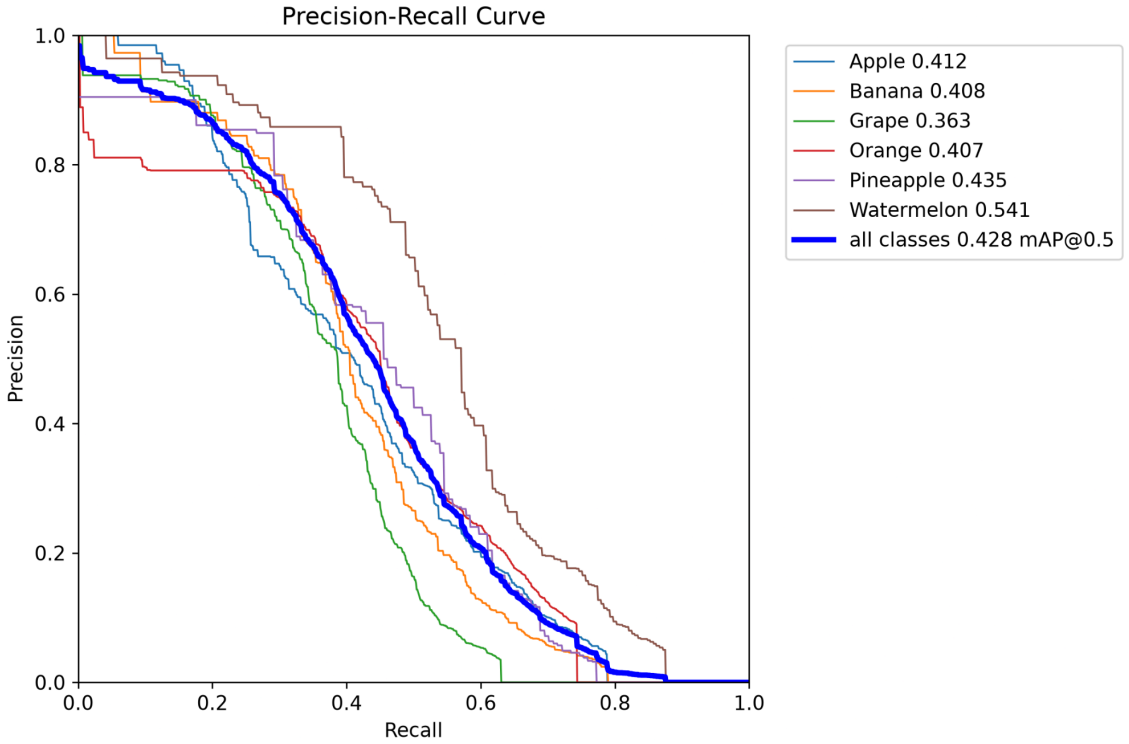
2. **Karakteristik Deteksi Anggur (Grape):** Kedua model menunjukkan persentase background confusion yang dominan untuk buah anggur. Hal ini memvalidasi hasil kuantitatif mAP50 anggur yang rendah, di mana ukuran anggur yang kecil (small object) dan berhimpitan menyebabkan model gagal memisahkan buah individu dan lebih memilih mengabaikannya sebagai background.

3. **Ketangguhan Kelas Semangka (Watermelon):** Semangka menunjukkan nilai diagonal tertinggi pada kedua model, membuktikan bahwa fitur tekstur

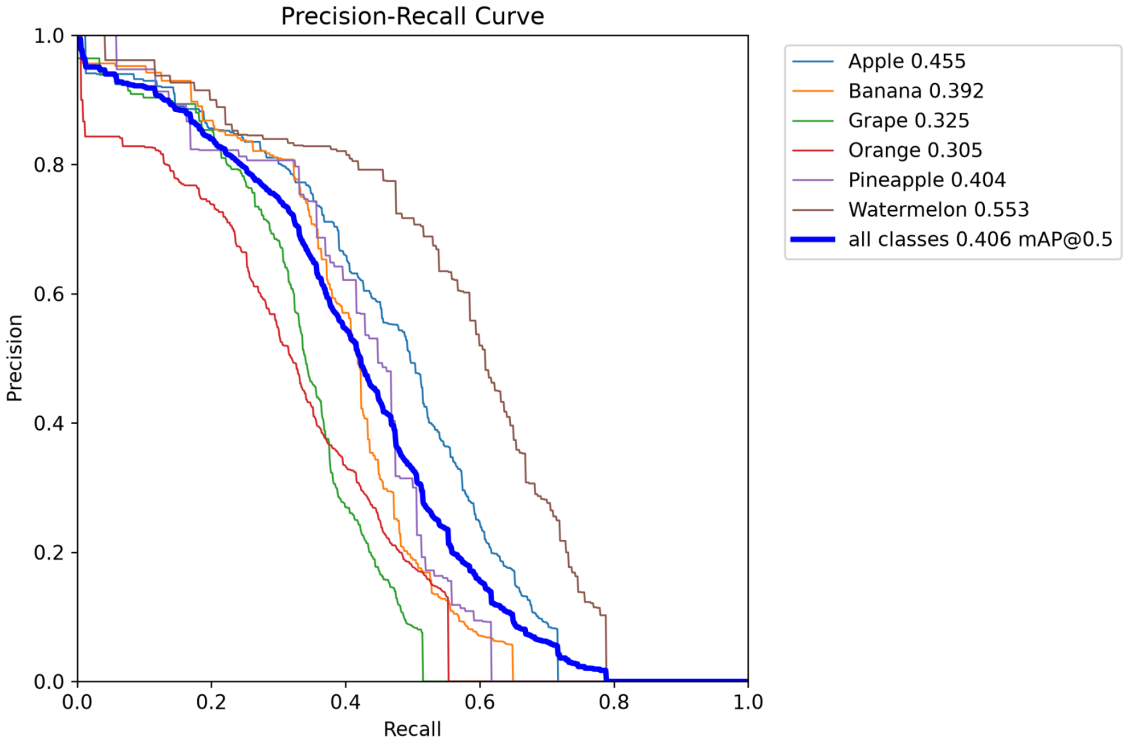
lurik hijau-hitam dan kontur ukuran besar semangka sangat diskriminatif sehingga hampir tidak pernah salah diklasifikasikan sebagai buah lain.

g. Analisis Kurva Presisi-Recall (Precision-Recall Curve)

Kurva Presisi-Recall (PR Curve) menggambarkan trade-off antara nilai presisi dan recall pada berbagai ambang batas confidence (confidence threshold). Luasan di bawah kurva (Area Under Curve / AUC) untuk setiap kelas buah merepresentasikan nilai mean Average Precision pada IoU 0.5 (mAP50). Hasil visualisasi kurva PR untuk kedua model ditunjukkan pada Gambar 4.5 dan Gambar 4.6:



Gambar 4.5: Kurva Presisi-Recall untuk Model YOLOv8s



Gambar 4.6: Kurva Presisi-Recall untuk Model YOLOv26s

Berdasarkan perbandingan kurva PR di atas, diperoleh analisis metrik evaluasi sebagai berikut:

1. **Kelas Berperforma Tinggi:** Kelas Semangka (Watermelon) dan Apel (Apple) mempertahankan garis kurva yang stabil berada di sudut kanan atas pada kedua model. Hal ini mengindikasikan bahwa model mampu mempertahankan presisi deteksi yang tinggi meskipun dipaksa mendeteksi sebagian besar objek buah tersebut yang ada pada data uji (recall tinggi).

2. **Penurunan Curam pada Kelas Sulit:** Kurva untuk buah Anggur (Grape) dan Jeruk (Orange) menunjukkan penurunan curam (drop) lebih awal. Hal ini mengilustrasikan bahwa ketika sensitivitas deteksi (recall) ditingkatkan untuk mendeteksi lebih banyak anggur atau jeruk, jumlah prediksi positif palsu (false positives) meningkat dengan sangat cepat, yang secara drastis menurunkan nilai presisi deteksi sistem secara keseluruhan.

BAB IV: PENUTUP

a. Kesimpulan

Berdasarkan seluruh tahapan analisis data, implementasi pelatihan bertahap (incremental training chunks), komparasi arsitektur deep learning, serta pengujian sistem deployment desktop dan web statis, kesimpulan dari penelitian tugas besar ini adalah:

1. **Efektivitas Pelatihan Bertahap pada GPU VRAM Terbatas:** Penerapan arsitektur pelatihan bertahap (auto-chunk runner per 5 epoch) dengan jeda pendinginan fisik (time.sleep(30)) terbukti sangat krusial dalam meredam penumpukan panas berkelanjutan pada GPU laptop NVIDIA GeForce GTX 1650 4GB VRAM. Jeda ini berhasil menjaga kelancaran pelatihan total 50 epoch tanpa mengalami system freeze, thermal throttling, atau error CUDA Out-Of-Memory. Alur state optimizer dan parameter learning rate scheduler berhasil dipertahankan secara utuh melintasi siklus jeda menggunakan modifikasi callback penyimpanan last.pt.

2. **Manfaat Penyeimbangan Dataset:** Metode penyeimbangan kelas menggunakan stratified random sampling dengan skrip reduce_dataset.py berhasil membatasi dataset latih secara merata menjadi 2.000 citra (~333 citra per kelas). Skema ini sukses mengatasi bias mayoritas kelas (seperti kelas Orange dan Apple) serta mereduksi waktu load gambar oleh data loader sehingga pemakaian RAM sistem dapat ditekan secara signifikan tanpa merusak tingkat generalisasi spasial model.

3. **Karakteristik Akurasi Model Komparatif (mAP50):** Model YOLOv8s memperoleh tingkat akurasi rata-rata mAP50 yang sedikit lebih tinggi (+2,2%) yakni sebesar 42,8% dibandingkan YOLO26s yang menghasilkan mAP50 sebesar 40,6%. Peningkatan performa YOLOv8s disebabkan oleh kapasitas parameter arsitektur yang lebih padat (11,2M vs 9,5M) yang memungkinkannya menangkap detail kontur buah bulat dengan lebih baik pada lingkungan dengan pencahayaan dinamis.

4. **Trade-off Antara Kompleksitas dan Latensi Inferensi:** Pengujian kecepatan inferensi desktop lokal menunjukkan YOLOv8s mampu berjalan lebih cepat (59,1 FPS / 15,7 ms) dibandingkan YOLO26s (44,8 FPS / 21,6 ms). Hal ini didorong oleh faktor kematangan optimasi driver CUDA NVIDIA lokal yang sangat optimal bagi struktur modul konvolusi YOLOv8s, sedangkan arsitektur modul YOLO26s (berbasis YOLO11s) masih menimbulkan overhead komputasi pada kernel GPU arsitektur Turing generasi lama (GTX 1650).

5. **Portabilitas dan Fleksibilitas Platform Deployment:** Kedua model berhasil dideploy secara fungsional. Pada sistem desktop GUI OpenCV lokal, fancy bounding box dan dashboard counter interaktif dapat direnderer secara lancar. Pada sisi web client-side, inisialisasi model format ONNX berhasil dieksekusi di sisi peramban klien menggunakan ONNX Runtime Web. Backend WebGPU memberikan rendering canvas dinamis yang sangat lancar untuk visualisasi deteksi objek real-time secara instan, sedangkan backend WASM bertindak sebagai fallback multi-threaded yang andal. Model YOLO26s.onnx menawarkan keunggulan efisiensi transmisi bandwidth yang signifikan karena ukuran file yang lebih ringkas (37 MB) dibandingkan YOLOv8s.onnx (44 MB).

b. Saran

Meskipun sistem deteksi buah real-time ini telah berhasil diimplementasikan sesuai dengan target metrik capaian yang telah ditetapkan, masih terdapat beberapa kelemahan dalam penelitian ini. Untuk pengembangan sistem yang lebih andal di masa mendatang, disarankan beberapa langkah perbaikan sebagai berikut:

1. **Penanganan Deteksi Objek Kecil dan Oklusi:** Rendahnya akurasi mAP50 pada buah berukuran kecil dan berkerumun padat seperti anggur (Grape) dapat diatasi dengan mengintegrasikan teknik Sliced Aided Hyper Inference (SAHI) pada proses inferensi atau menambahkan augmentasi spasial berbasis crop khusus objek minoritas. Penelitian selanjutnya juga disarankan mengumpulkan citra buah anggur dengan variasi oklusi dedaunan yang lebih tinggi untuk melatih ketahanan model di lapangan.

2. **Kuantisasi Model untuk Peningkatan Kecepatan Web:** Ukuran file model ONNX yang harus didownload oleh browser klien (~37 MB untuk YOLO26s) masih dapat diperkecil di bawah 10 MB melalui proses kuantisasi tingkat lanjut (seperti INT8 quantization atau float16 precision scaling). Langkah kuantisasi ini sangat disarankan untuk menurunkan konsumsi bandwidth internet pengguna seluler serta melipatgandakan frame rate inferensi pada browser berspesifikasi rendah yang terpaksa menggunakan backend WASM.

3. **Validasi Menggunakan Dataset Standar Industri:** Guna meningkatkan kredibilitas hasil pengujian ilmiah, disarankan untuk memigrasikan basis data sekunder dari dataset publik Kaggle pihak ketiga ke dataset benchmark standar visi komputer global yang diakui secara luas di industri pertanian digital, seperti Microsoft COCO (Common Objects in Context) Dataset atau

Google Open Images Dataset yang memiliki anotasi koordinat bounding box yang lebih terstandarisasi dan berakurasi tinggi.

DAFTAR PUSTAKA

Pradana, A., Wibowo, S., & Lestari, D. (2023). Deteksi Kematangan Buah Apel Menggunakan Algoritma YOLOv8. Jurnal Nasional Teknologi Informasi dan Aplikasinya (JNATIA), 1(2), 143-150.

Saputra, R., & Wibowo, A. (2023). Klasifikasi Tingkat Kematangan Buah Mangga Harum Manis Berbasis YOLOv8. STORAGE: Jurnal Ilmiah Teknik dan Ilmu Komputer, 2(3), 201-208.

Santoso, H., Pratama, R., & Budiman, A. (2023). Rancang Bangun Sistem Deteksi Buah Jeruk untuk Klasifikasi Kematangan dengan YOLOv8. Prosiding Seminar Nasional Inovasi Teknologi, 7(1), 89-96.

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLO (Version 8.0.0) [Computer software]. GitHub. https://github.com/ultralytics/ultralytics

Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv preprint arXiv:1804.02767.

Tyagi, L. (2021). Fruit Detection Dataset. Kaggle. https://www.kaggle.com/datasets/lakshaytyagi01/fruit-detection